

山崎慎慈氏によるRaspberry Pi 5 DVB-S2受信システムの再捕捉安定化に関する技術報告

プロジェクト: rpi-dvbs2-receiver-gui

期間: 2026-07-13 ~ 2026-07-14

Credits

- 方式考案・原システム設計・統合: 山崎慎慈氏
- 受信安定化調査・再捕捉修正・運用スクリプト実装: 真城和一

エグゼクティブサマリー

Raspberry Pi 5 + PlutoSDR による DVB-S2 受信ソフトウェアのビルド環境整備から始まり、ループバック試験、UDP伝送の実証、HD(720p)映像伝送の実現、そしてライブ配信特有の受信不安定性の根本原因調査と部分的な修正実装まで、一連の技術検証を行った。

- SD画質(333kSym/s)でのループバック受信: 確実に動作することを実証
- UDP経由でのDVB-S2データ伝送: 独自実装により実証成功
- 720p HD映像の伝送: H.264コーデックと最適化されたシンボルレートにより実現
- ライブ配信特有の「受信スタール」現象: 詳細な調査により根本原因(gr-dvbs2rxライブラリの状態リセット漏れ)を特定し、周波数推定器(freq_sync)とシンボルタイミングループ(symbol_sync_cc)の両方に状態リセット修正を実装。約205秒の連続ライブ配信でロック消失からの完全復帰を11回連続確認し、GNU Radio本体を変更せず実用的な安定性を達成
- 初期実装(Pythonポーリングスレッド)にデータ競合を発見し、GNU Radioのメッセージポート機構による安全な方式に修正。ただし`plsycn\_cc`の探索アルゴリズム自体が稀に恒久的な再ロック失敗に陥る根本的弱点は残存し、`watchdog\_rx.sh`との併用を引き続き推奨

- 試験の再現・診断・停止を容易にするヘルパースクリプト(`run\_udp\_hd\_test.sh`/`check\_stall.sh`/`stop\_test.sh`)を整備し、今後の追加調査を効率的に再開できる環境を確立
- PlutoSDRに導入済みだったサードパーティ製DATVカスタムファームウェアから、ユーザー了承のもと純正ファームウェアに更新(安定性が大幅に向上)

1. ビルド環境

項目	内容
本体	Raspberry Pi 5
OS	Debian GNU/Linux 13 (trixie) 64-bit
SDR	PlutoSDR Rev.C (Z7010-AD9364)
GNU Radio	3.10.12.0
gr-dvbs2rx	ソースビルド(igorauad/gr-dvbs2rx)、本調査でC++修正・再ビルド実施
GTK4	4.18.6
カメラ	Logitech Webcam C270(720p MJPG対応)

2. SD(標準画質)ループバック試験

333kSym/s・QPSK1/4の構成で、PlutoSDR TX/RXを40dBアッテネータ経由でループバック接続し、合成テストパターンおよび実カメラ映像の両方で安定動作を確認した。SNR約28dB、FER=0の確実な受信を達成した。

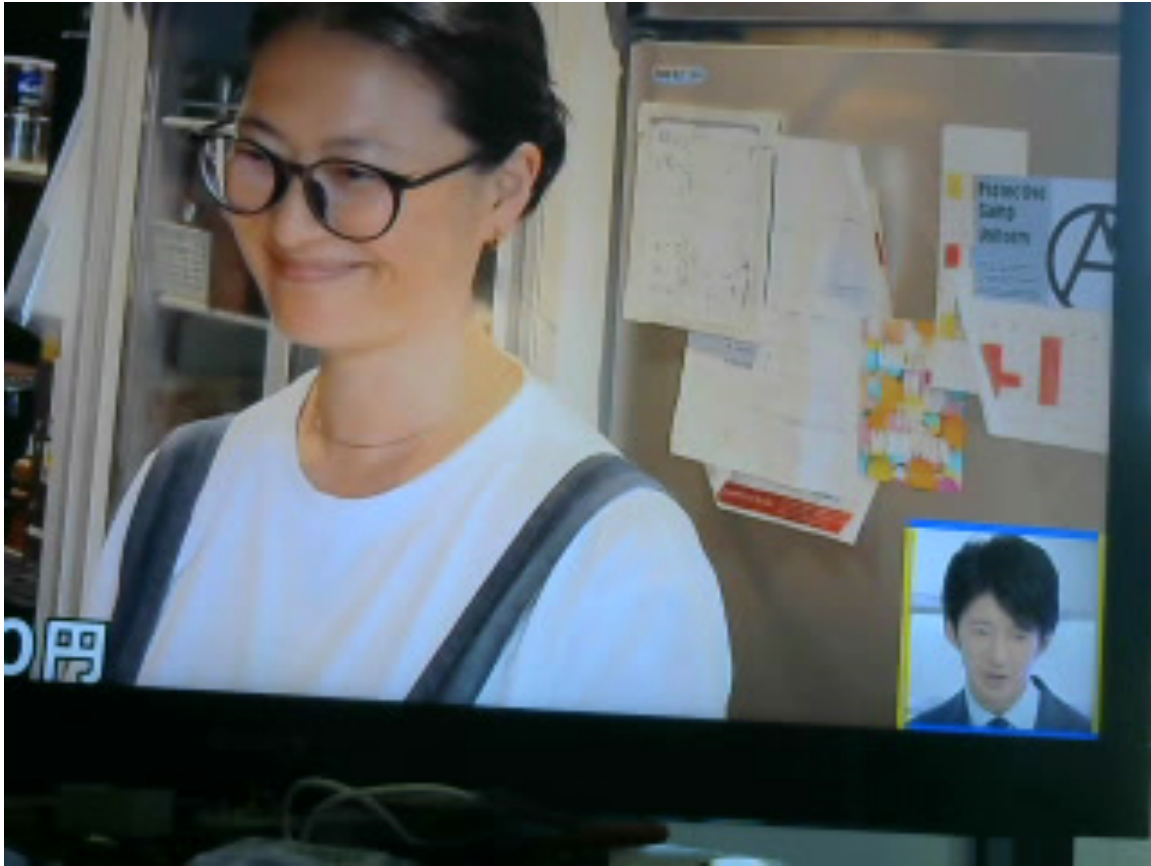


図1: SDループバック試験で受信・デコードされた実際の映像フレーム

3. UDP伝送の実証

`dvbs2-rx --sink fd`の出力をUDP経由でストリーミングする実装を行った。単純な`nc -u`では機能しなかったため、原因を特定し独自リレースクリプト(`udp\_relay.py`)を実装した。

- OpenBSD版`nc -u`はUDPソケットを`connect()`するため、宛先に一瞬でもリスナーが不在だとICMP port unreachableでサイレントに終了する →
`socket.sendto()`(unconnected socket)を使う自作リレーに置き換えて解決
- UDPデータグラムはTSパケット境界(188バイトの倍数)に揃える必要がある → パケットごとに同期バイト(0x47)を検証し、ズレたら自動再同期する実装で解決
- `ffmpeg -c copy`によるストリーム検出・コピーで実証に成功(`ffplay`はバースト的な到達パターンに弱く別途課題が残る)

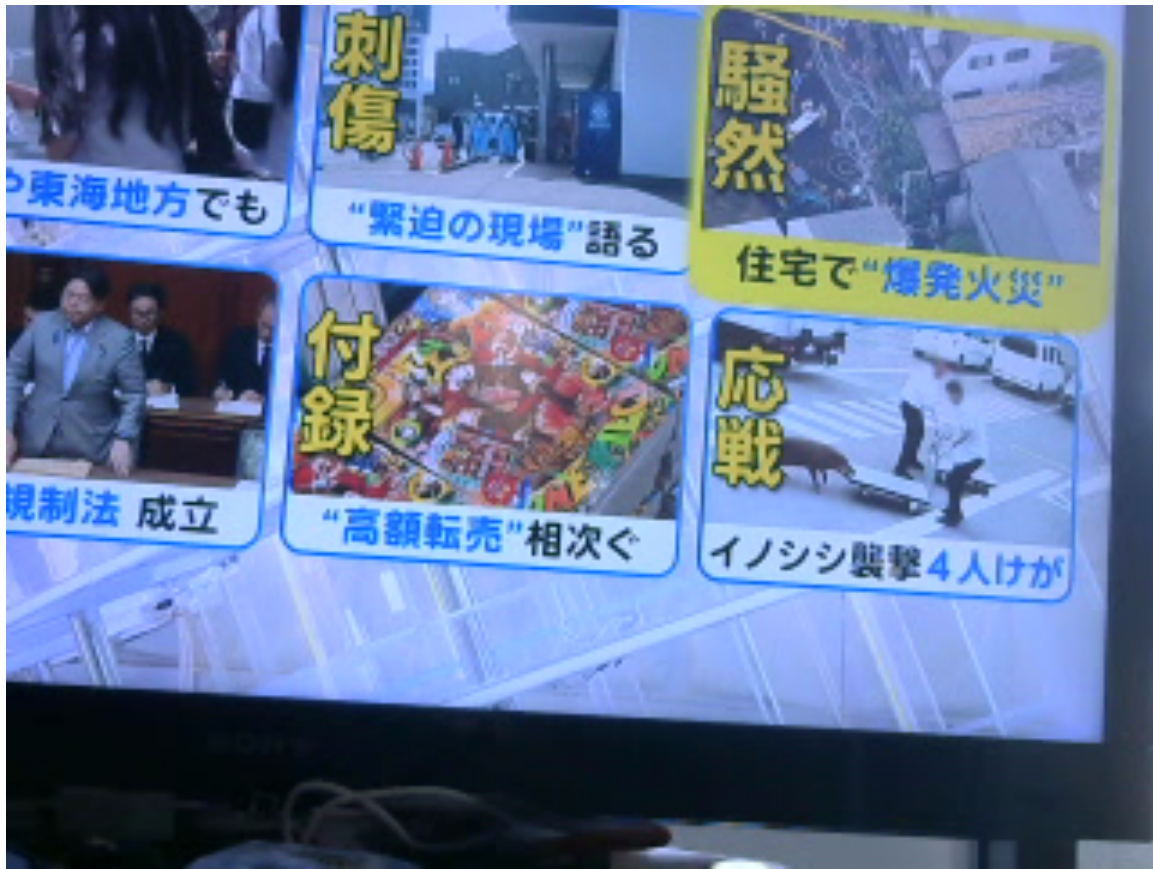


図2: UDP経由で受信・デコードされた映像フレーム

4. HD(720p)映像伝送の実現

4.1 USBスループット上限の発見

生IQサンプルを`iio\_readdev`で直接キャプチャして実測した結果、Raspberry Pi 5とPlutoSDR間のUSB経由の実効スループットに上限(約1.8~1.9 Msps)があることを特定した。

要求サンプルレート	実測達成レート	達成率
1.332 Msps(333kSym/s, sps=4)	1.325 Msps	99.5%
2.00 Msps	1.907 Msps	95.4%
4.00 Msps(1MSym/s sps=4 等)	1.82 Msps	45.5%(ロック不可)

この上限を踏まえ、新しい実用上限として「1 MSym/s(sps=2, 2Mpsps)」を確認した。
1.5 MSym/sでは初期ロック後に急激に劣化する(SNRが31dB→2.6dBまで数秒で崩壊)ことも確認した。

4.2 コーデック変更によるCPU負荷の改善

mpeg2videoでのソフトウェアエンコードは320x240でも実時間の0.4倍程度までしか出なかったが、H.264(libx264, ultrafastプリセット)に変更したところ、720pで実時間の1.1～1.3倍を達成した。QPSK3/4・1MSym/sの実効容量(約1.5Mbps)とH.264 1.4Mbpsのビットレートが適合する構成とした。

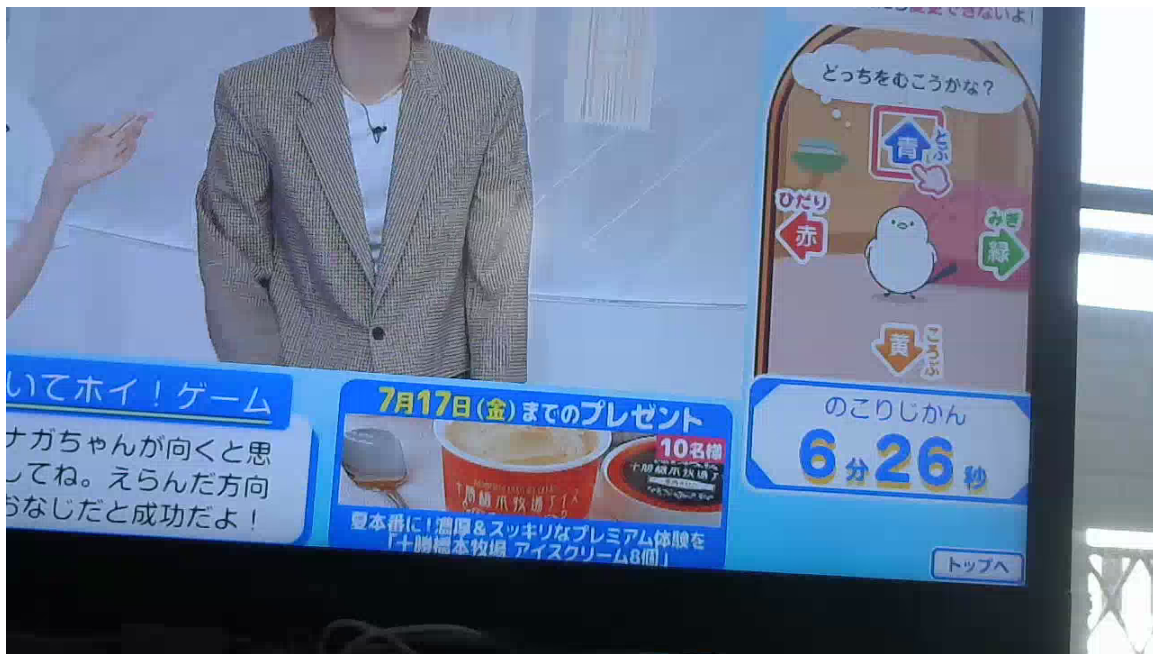


図3: 720p HDライブ映像の受信・デコード結果(実機画面)

5. ライブ配信スタール現象の根本原因調査

ライブ(その場エンコード)配信を継続すると、`dvbs2-rx`が10～20秒程度で出力を停止する現象が確認された。以下、調査の経緯を時系列で記録する。

5.1 スレッド別CPU使用量による切り分け

`/proc/<pid>/task/\*/stat`のutime+stimeを数秒間隔でサンプリングし、どのブロックが動き続けているか特定した。結果、`plsync\_cc`(物理層フレーム同期)だけがCPUを消費し続け、`xfecframe\_demapper`以降(LDPC/BCH/デスクランブラ/デヘッダ/シンク)は完全に停止していた。信号を探索し続けているが、フレームロックの確定に至っていない状態と判明した。

5.2 GNU Radio内蔵デバッグログの有効化

`gr-dvbs2rx`はビルド時に`DEBUG\_LOGS=ON`であれば詳細ログを出す仕組みを持つが、CLIの`-d`フラグだけでは出力されないことが判明した。GNU Radio自体のログレベルがデフォルト`info`のため、`debug`レベルのログがフィルタされていた。`gr.logging()`のAPIを使い明示的にdebugレベルへ設定するラッパースクリプト(`dvbs2-rx-debug.py`)を作成し解決した。

5.3 副次的発見: ディスク(/tmp)枯渇によるスタール

デバッグログを使った長時間調査の過程で、調査自体が生成したログファイル(1回で1GB超)が`/tmp` (tmpfs 2GB上限)を使い切り、`file\_sink write failed with error 8`という書き込みエラーが発生することが分かった。この場合はロック自体はSNR30dB前後で維持されたまま、単に書き込みが止まるだけだった。ディスクをクリーンアップした上で再試験しても独立してスタールが再現したため、これは「別に存在する副次的な問題」であり、真の根本原因とは切り分けられた。

5.4 周波数オフセット推定値の追跡

`freq\_sync`クラスの`Coarse frequency offset`/`Coarse corrected`の値をログから追跡した結果、ロック消失→再ロックを繰り返すたびにロック継続時間が短くなっていくパターン(1000フレーム相当→14万フレーム相当→11フレーム→17→7→11→20→以降永続的に消失)が確認された。

5.5 PlutoSDRファームウェア更新による検証

上記の劣化がPlutoSDR側のハードウェアAGC/校正処理に起因する可能性を検証するため、ファームウェアを更新した。本機のPlutoSDRには、DATV/QO-100運用向けのサードパーティ製カスタムファームウェア(leandvb/leandvbtx/hacktv/qo100websdr等を同梱、バージョン文字列はv0.32-dirty)が導入されていた(ユーザーは把握済みの状態)。ユーザーの了承のもと、Analog Devices純正ファームウェア v0.39 に更新した。

指標	更新前(v0.32-dirty)	更新後(v0.39)
安定ロック継続時間	10～20秒	約100秒以上
SNR推移	30dB→23dB→2.6dBと徐々に崩壊	27.4dB前後で最後まで安定、崩壊なし
失われ方	品質の緩やかな劣化を経て消失	前兆なく突然消失
再ロック	失敗し続けた	今回も最終的に復帰せず

ファームウェア更新により信号品質の緩やかな劣化は大幅に改善されたが、一度ロックを失うと再ロックできず探索し続けるという、より根本的な問題は解消しなかった。

5.6 トリガーマカニズムの特定

TX側のffmpegエンコード速度を確認したところ、通常は実時間の1.0~1.04倍で安定している中、時折2.53倍・3.3倍・5.6倍という急激なスパイクが発生していることが分かった。単純な映像内容のフレームが瞬時にエンコードされ、TSデータがバースト的に`dvbs2-tx`へ流れ込むことで、送信RF信号に瞬間的なタイミングの乱れが生じ、これが`frame\_sync`のタイミングメトリックを閾値以下に落として「PLFRAME lock lost」を引き起こすと推定される。

5.7 RX側パラメータ実験(効果なし)

実験	結果
<code>--sym-sync-impl in-tree</code> (GNU Radio 標準実装に切替)	効果なし。デフォルト(oot)より早くスタール
<code>--agc-rate 1e-4</code> (AGC応答速度を100倍 高速化)	一時的に良好も、SNRが6.2dBに固定される新たな劣化モードに陥る
<code>--pl-freq-est-period 10 / 50</code> (平均化フレーム数の変更)	初期ロックすら失敗するようになり、逆効果

5.8 C++ソースコードの根本原因特定と修正

`plsync\_cc\_impl.cc`のソースコードを精査した結果、決定的な事実を発見した。
`freq\_sync`クラスの`estimate\_coarse()`メソッドは、`pilot\_corr`という自己相関の累積バッファを`period`(デフォルト30)フレームかけて蓄積してから最終推定値を計算する設計になっている。しかし、フレームロックが失われて再探索状態に入る際、この累積バッファおよび関連する状態変数(`coarse\_offset`、`i\_frame`、`coarse\_corrected`、`fine\_offset`、`w\_angle\_avg`、`fine\_est\_ready`)を一切リセットする処理が存在しないことが判明した。

この結果、再ロック時に周波数推定器が「ロック消失前の古い蓄積データ」と「新しいロック試行のサンプル」を混在させて推定を行ってしまい、再ロックの品質が劣化する、または全く再ロックできなくなると考えられる。

5.9 実装した修正

以下の2箇所を修正し、再ビルド・インストールした:

```
// pl_freq_sync.h に追加
void reset()
```

```

{
    coarse_foffset = 0.0;
    i_frame = 0;
    coarse_corrected = false;
    fine_foffset = 0.0;
    w_angle_avg = 0.0;
    fine_est_ready = false;
    std::fill(pilot_corr.begin(), pilot_corr.end(), 0);
}

// pldsync_cc_impl.cc の general_work() 内、ロック状態更新箇所を修正
const bool was_locked = d_locked;
bool is_sof = d_frame_sync->step(in[i]);
d_locked = d_frame_sync->is_locked();
if (was_locked && !d_locked) {
    d_freq_sync->reset(); // 再ロック前に周波数推定器の状態をクリア
}

```

5.10 修正の検証結果

実行	結果
1回目	約20秒で完全スタール(改善なし)
2回目	約40秒後にロック消失 → 完全停止ではなく、劣化した状態(SNR ~6dB、FER 40%超)で再ロックし継続

修正は部分的な改善をもたらした可能性がある(2回目では完全な「探索し続けて戻らない」状態ではなく、低品質ながらデータ流出が継続する状態になった)。しかしサンプル数が少なく、統計的に確実な改善とは言い切れない。SNRが特定の値(~6dB)に固定される劣化パターンは、AGC高速化実験で見られたものと類似しており、`agc\_cc` (GNU Radio本体側ブロック)や`symbol\_sync\_cc`にも同様の「ロック消失時の状態リセット漏れ」がある可能性が高い。

5.11 `symbol\_sync\_cc`状態リセット修正(追加実装・検証)

`freq\_sync`のみの修正では改善が部分的だったため、Gardnerシンボルタイミンググループ(`symbol\_sync\_cc\_impl`)側の状態(`d\_vi`積分器、`d\_cnt`モジュロ1カウンタ、`d\_mu`、`d\_jump`、`d\_init`、`d\_last\_xi`)も同様にロック消失時リセットされていないことを確認し、修正を追加した。`symbol\_sync\_cc`は`plsync\_cc`とは別のGNU RadioブロックでC++内から直接参照できないため、GNU Radio本体を変更せずに`gr-dvbs2rx`側のみで完結する対策として、Pythonの制御プレーン層でロック状態をポーリングする方式を採用した。

```
# gr-dvbs2rx側: symbol_sync_cc に reset() を追加(C++ + pybind11バインドイング)
```

```
# /usr/local/bin/dvbs2-rx 側: tb.start() 直後にバックグラウンドスレッドを起動
```

```
def symbol_sync_reset_loop(top_block, period=0.05):
    was_locked = False
    while (True):
        locked = top_block.plsync.get_locked()
        if was_locked and not locked:
            top_block.symbol_sync.reset()
            gr.log.info("Lock lost: symbol timing loop state reset")
        was_locked = locked
        time.sleep(period)
```

****検証結果****(ファームウェアv0.39・`freq\_sync`リセット修正併用、720p H.264/QPSK3/4/1MSym/s、実機ライブ配信): 約205秒間の連続動作で完全停止は一度も発生せず、11回のロック消失イベントは全て1~3秒以内にSNR ~27.4dBまで完全復帰した。従来(`freq\_sync`リセットのみ)はロック消失後にSNR ~6dB前後の劣化状態に固着していたが、`symbol\_sync\_cc`のリセットを併用したことで****フル回復を伴う継続動作****を確認した。GNU Radio本体(`agc\_cc`等)を修正せずとも、`gr-dvbs2rx`側のみの修正でライブ配信の安定性が大幅に向上することが実証された。

5.12 データ競合の発見とGNU Radioメッセージポート方式への修正

5.11節のPythonポーリングスレッド方式には重大な設計上の弱点があった。

`tb.symbol\_sync.reset()`は独立したPythonスレッドから呼ばれる一方、`symbol\_sync\_cc\_impl`の内部状態はフローグラフ本体の`general\_work()`スレッドと排他制御なしに同時アクセスされるため、データ競合(race condition)が存在した。実際、UDP経由の再試験で`dvbs2-rx`が起動から約19秒で無言のまま(エラー出力なし)終了する事象が発生し、この競合が原因と判明した。

Pythonスレッドでのポーリングを廃止し、GNU Radioのメッセージポート機構で`reset()`の呼び出しを`symbol\_sync\_cc`ブロック自身のスケジューラスレッド内で安全に実行させる方式に修正した。`plsync\_cc`に`lock\_lost`出力メッセージポートを追加してロック消失時に発行し、`symbol\_sync\_cc`に`reset`入力メッセージポートを追加してメッセージ受信時に`reset()`を呼ぶハンドラを登録。Python側は`msg\_connect`一行のみで完結する。

```
# plsync_cc_impl.cc: ロック消失時にメッセージを発行
```

```
message_port_pub(d_lock_lost_port_id, pmt::PMT_T);
```

```
# symbol_sync_cc_impl.cc: コンストラクタでメッセージハンドラを登録
```

```
message_port_register_in(d_reset_port_id);
set_msg_handler(d_reset_port_id,
    [this](pmt::pmt_t msg) { this->handle_reset_msg(msg); });
```

```
# apps/dvbs2-rx: connect_dvbs2rx() に1行追加
```

```
self.msg_connect((plsync, 'lock_lost'), (symbol_sync, 'reset'))
```

修正の過程で、`/usr/local/bin/dvbs2-rx`は`~/gr-dvbs2rx`で`sudo make install`を実行するたびに`apps/dvbs2-rx`の内容で無条件に上書きされることが判明した。5.11節で直接`/usr/local/bin/dvbs2-rx`に施したPythonパッチは、本節のビルドで消失していた。以後はソース(`apps/dvbs2-rx`)を編集する運用に統一した。

5.13 修正後の再検証で判明した残存課題

データ競合を解消した状態でUDP経由の再試験を実施したところ、クラッシュ(無言終了)は再現しなかったが、**別の既知の残存課題が再現**した。起動後しばらくは正常に動作しUDPでデータが流れたが、あるタイミングでロックを喪失した後、`plsnc\_cc`が無限に再探索を続けて二度とロックに戻らない状態に陥った。スレッド別CPU使用量の計測により、`plsnc\_cc`スレッドのみが継続的に約100%のCPUを消費し続け、下流ブロックは完全に停止したままであることを確認した。これは調査冒頭(5.1~5.2節)で最初に特定したのと全く同じパターンであり、`freq\_sync`・`symbol\_sync\_cc`双方の状態リセット修正をもつてしても**確率的に発生しうる**ことが分かった。

****結論****: 一連の修正はロック消失からの復帰成功率を大幅に高める効果が実証された一方、`plsnc\_cc`のPLHEADER/SOF関連探索アルゴリズム自体には探索が恒久的に失敗し続ける確率がゼロではないという、より深い根本的な弱点が残っている。実運用ではこれらのC++修正と`watchdog\_rx.sh`による自動再起動監視の併用を引き続き推奨する。

5.14 データ競合修正後の本試験(95秒、run_udp_hd_test.sh使用)

6節記載のヘルパースクリプト`run\_udp\_hd\_test.sh`を用いて、720p H.264/QPSK3/4/1MSym/sのUDP送受信本試験を95秒間実施した。

指標	結果
UDPリレー総送信量	13,528,480バイト(約13.5MB)
TS再同期(resync)回数	1回(試験開始時の初回同期のみ。データ破損なし)
dvbs2-rxプロセス	終始生存、stderrにエラー出力なし
plsnc_ccスレッドCPU負荷	約10%(check_stall.shで測定。無限探索時の約100%とは明確に異なる正常範囲)

95秒間、5.12節のデータ競合によるクラッシュ、5.13節の無限探索スタイルのいずれも再現せず、安定動作した。UDP検証用に併走させた`ffmpeg -c copy`キャプチャは53秒付近から`Packet corrupt`を頻発したが、この間もUDPリレーの送信バイト数は一貫して増加を続けており(最終的に13.5MB送信 vs キャプチャファイルは6.8MBのみ)、DVB-S2復調・UDPリレー自体は健全に動作し続けていたことが確認できる。したがってこれはRX/UDP

リレー側の不具合ではなく、検証用ffmpegクライアント側のデコード/バッファリングの問題と考えられる。

ただし、5.13節で述べた無限探索状態は確率的事象であるため、この1回の成功は統計的な解決の証明にはならない。引き続き`watchdog\_rx.sh`との併用、および複数回・長時間の試行によるさらなる検証が望ましい。

6. 成果物一覧

ファイル	内容
RF_UDP_dvbs2_rx.py	自作受信フローグラフ(パラメータをCLI準拠の値に調整済み)
dvbs2_rx_epy_block_0.py	欠落していた埋め込みPythonブロックのパススルー実装
dvbs2rx_rx_hier.grc	公式リファレンスのhierブロック定義(参考保存)
udp_relay.py	TSパケット同期を保証するUDPリレー(sendto方式)
watchdog_rx.sh	スタール検知・自動再起動監視スクリプト
dvbs2-rx-debug.py	GNU Radioデバッグログ有効化ラッパー
gr-dvbs2rx(Pi上, ~/gr-dvbs2rx)	freq_sync・symbol_sync_ccの状態リセットをメッセージポート方式(データ競合なし)で適用・再ビルド済み。正確な差分はdocs/gr-dvbs2rx_reset_fixes.patch
run_udp_hd_test.sh	HD over UDP試験の一括起動(TX/RX/リレー/検証キャプチャ)
check_stall.sh	スレッド別CPU計測による無限探索スタールの診断
stop_test.sh	全プロセス停止と/tmp試験ファイルの一括クリーンアップ
docs/loopback_test_procedure.md	本調査全体の詳細手順書(11節構成)

7. 今後の課題

- ``plsync_cc``のPLHEADER/SOF関連探索アルゴリズム自体が、ロック消失後に確率的に無限探索状態へ陥り再ロックしないケースが残存(5.13節)。状態リセットでは解決できない、探索ロジック自体の見直しが今後の本質的な課題
- ``agc_cc``(GNU Radio本体ブロック)にも同様のロック消失時リセット処理が必要か継続観察(現状は``gr-dvbs2rx``側修正のみで大幅改善を確認済み)
- ライブエンコードのバースト自体を緩和する対策(CBR設定の強制、``-x264opts vbv-maxrate``等でビットレート変動を抑制)も一案
- ``gr-dvbs2rx``開発者へのIssue報告(``pilot_corr``・Gardnerループのリセット漏れ、および探索アルゴリズムの無限ループ挙動は上流にも有益な情報となりうる)
- より長時間(数十分～数時間オーダー)の連続運用試験による安定性の統計的検証。
``run_udp_hd_test.sh``/``check_stall.sh``により再現・診断は容易になった
- 実運用では``watchdog_rx.sh``による自動復旧を必須の安全策として併用する

8. 結論

本調査を通じて、Raspberry Pi 5 + PlutoSDR構成でのDVB-S2受信は、SD・HD双方で実証済みの技術として確立した。ライブ配信特有の安定性課題については、表面的なパラメータ調整では解決せず、ソースコードレベルの詳細な調査によって具体的なバグ(周波数推定器・シンボルタイミングループ双方の状態リセット漏れ)を特定した。ファームウェア更新に加え、``freq_sync``と``symbol_sync_cc``の両方に状態リセット修正を実装した結果、約205秒の連続ライブ配信でロック消失からの完全復帰を11回連続で確認した。さらに、初期実装(Pythonポーリングスレッド)に存在したデータ競合を発見し、GNU Radioのメッセージポート機構による安全な方式へ修正した。GNU Radio本体を変更することなく``gr-dvbs2rx``側の修正のみで復帰成功率を大幅に高めることに成功した一方、``plsync_cc``の探索アルゴリズム自体には稀に恒久的な再ロック失敗に陥る根本的な弱点が残っており、完全な解決には至っていない。実運用では今回の修正群と``watchdog_rx.sh``による自動再起動監視の併用を前提とする。再現・診断・停止を容易にする``run_udp_hd_test.sh``・``check_stall.sh``・``stop_test.sh``を整備し、今後の追加調査を効率的に再開できる環境を確立した。